



VIGOR

Visual Integrated Governance & Overlap Registry

Projektrapport



11. FEBRUAR 2026

GRUPPE 11

Indhold

1	Indledning.....	6
1.1	Baggrund	6
1.2	Projektets kontekst (Bostedet “Slottet”)	6
1.3	Problemstilling	6
1.4	Problemformulering.....	6
1.5	1.5 Formål.....	7
1.6	1.6 Læsevejledning.....	7
2	Business case	7
2.1	Forretningsmæssig baggrund	7
2.2	Forretningsmæssig problemstilling	7
2.3	Interessenter	8
2.4	Systemets brugere	8
2.5	Scenarieoversigt.....	8
2.5.1	0-scenariet: Fortsætte med Excel og stramme op.....	8
2.5.2	1-scenariet: Udvikling af digitalt overlapsystem (valgt løsning)	9
2.5.3	2-scenariet: Manuel forbedring af arbejdsgange uden nyt system.....	9
2.5.4	Valgt scenarie	9
2.5.5	Implementering og opfølgning	11
3	Scope og Afgrænsning	12
3.1	In Scope	12
3.2	Out of Scope	12
3.3	Antagelser.....	12
3.4	Begrænsninger	12
4	Metode og Proces	12
4.1	Overordnet udviklingsmetode.....	13
4.2	Iterativ og feature-baseret tilgang.....	13
4.3	Kombination af Scrum og XP-principper	13
4.4	Artefaktstrategi (levende diagrammer).....	13
4.5	Versionsstyring og commit-strategi	13

4.6	Definition of Done.....	14
5	Forretningsforståelse og Krav	15
5.1	Interessenter.....	15
5.2	Aktører.....	15
5.2.1	Personale.....	15
5.2.2	Vagtansvarlig.....	15
5.2.3	Leder	15
5.2.4	Superbruger	15
5.3	Overordnet forretningsproces (vagtskifte).....	15
5.4	Use case Backlog	15
5.4.1	UC01 – Log ind	15
5.4.2	UC02 – Se borgere på afdeling	16
5.4.3	UC03 – Opdatér borgerstatus / risikoprofil	16
5.4.4	UC04 – Registrér fast medicin	16
5.4.5	UC05 – Registrér PN medicin	16
5.4.6	UC06 – Se historik / audit-log.....	16
5.4.7	UC07 – Vælg vagt	17
5.4.8	UC08 – Se overlap.....	17
5.4.9	UC09 – Opret note	17
5.4.10	UC10 – Opret og afslut opgave	17
5.4.11	UC11 – Tildel personale til borger.....	17
5.4.12	UC12 – Tildel telefon til personale	17
5.4.13	UC13 – Administrér brugere og roller	18
5.4.14	UC14 – Public anonym oversigtsskærm	18
5.4.15	UC15 – Statistik og rapporter	18
5.5	Use Case Prioriteringstabel	18
5.6	Use Case Diagram	20
5.7	Brief Use Cases	21
5.8	UC1 – Login.....	Fejl! Bogmærke er ikke defineret.
5.9	UC2 – Vælg vagt.....	Fejl! Bogmærke er ikke defineret.

5.10	UC3 – Opdatér risikoprofil	Fejl! Bogmærke er ikke defineret.
5.11	UC4 – Registrér fast medicin.....	Fejl! Bogmærke er ikke defineret.
5.12	UC5 – Registrér PN medicin.....	Fejl! Bogmærke er ikke defineret.
5.13	UC6 – Opret note/opgave	Fejl! Bogmærke er ikke defineret.
5.14	UC7 – Se historik	Fejl! Bogmærke er ikke defineret.
5.15	UC8 – Tildel personale	Fejl! Bogmærke er ikke defineret.
5.16	Fully Dressed Use Cases	Fejl! Bogmærke er ikke defineret.
5.17	Registrér fast medicin	Fejl! Bogmærke er ikke defineret.
5.18	Opdatér risikoprofil.....	Fejl! Bogmærke er ikke defineret.
5.19	Vagtskifte.....	Fejl! Bogmærke er ikke defineret.
6	Domænemodel og Analyse	21
6.1	Domænemodel (Analysemodel)	21
6.2	Begrebsafklaring.....	22
6.3	Analyseovervejelser	22
7	Systemsekvens og Interaktionsanalyse	22
7.1	System Sequence Diagram – Medicin.....	22
7.2	System Sequence Diagram – Vagtskifte	22
7.3	Analyse af systemgrænse	22
8	Design og Arkitektur	22
8.1	Overordnet arkitektur	22
8.2	Valg af teknologi (.NET MAUI Blazor Hybrid)	23
8.3	MVC-struktur.....	23
8.4	Dependency Injection	23
8.5	SOLID, GRASP, KISS, DRY	23
9	Designmodeller	23
9.1	Design Class Diagram (DCD)	23
9.2	Sekvensdiagram – Vagtskifte (design-level).....	24
9.3	Sekvensdiagram – Borgerstatus	24
9.4	Ansvarsfordeling i klasser	25
10	Datamodel	26

10.1	ER-diagram	26
10.2	Entiteter og relationer	26
10.3	Historik- og audit-strategi (append-only)	26
11	Brugergænseflade.....	26
11.1	Designprincipper	26
11.2	Wireframe – Public View	26
11.3	Wireframe – Login	26
11.4	Wireframe – Overlap screen	26
11.5	Brugervenlighed for IT-ukyndige	26
12	Medicinhåndtering	27
12.1	Aktivitetsdiagram – Fast medicin.....	27
12.2	PN medicin-flow	27
12.3	Validering og logning	27
13	Ikke-funktionelle Krav (Supplementary Specification)	27
13.1	Brugervenlighed	27
13.2	Sikkerhed	27
13.3	Performance	27
13.4	Tilgængelighed	27
13.5	Vedligeholdbarhed	27
14	Risikovurdering og GDPR	27
14.1	GDPR-kontekst.....	27
14.2	Risikotabel	27
14.3	Anonymisering vs. lukket system	27
14.4	Rollebaseret adgang	27
14.5	Audit-log og sporbarhed	27
14.6	Argumentation for løsning	27
15	Deployment og Infrastruktur	28
15.1	Deployment diagram	28
15.2	Lukket installationsscenario	28
15.3	Ekstern anonym adgang	28

15.4	Driftsovervejelser	28
16	Implementering	28
16.1	Projektstruktur	28
16.2	Mappestruktur og namespaces	29
16.3	Centrale services.....	29
16.4	Teststrategi.....	29
16.5	Eksempler på kode	29
17	Test og Kvalitet	29
17.1	Unit tests	29
17.2	Integrationstest	29
17.3	Manuel test	29
17.4	Kvalitetssikring	29
18	Refleksion	29
18.1	Metoderefleksion.....	29
18.2	Samarbejde og iterationer	29
18.3	Hvad fungerede godt.....	29
18.4	Hvad kunne forbedres	29
19	Konklusion	29
19.1	Opfyldelse af problemformulering	29
19.2	Fremtidige udvidelser	29
20	Referencer.....	30
21	Bilag.....	31
21.1	Ekstra diagrammer.....	31
21.2	Ekstra use cases.....	31
21.3	Screenshots.....	33
21.4	GitHub link.....	33

1 Indledning

1.1 Baggrund

1.2 Projektets kontekst (Bostedet “Slottet”)

1.3 Problemstilling

Bostedet “Slottet” bruger i dag et Excel-ark til at håndtere overlap mellem vagter. Det fungerer i praksis, men det er ikke lavet til formålet. Der bliver skrevet noter, registreret medicin og opdateret borgerstatus, men der er ingen rigtig struktur, ingen sporbarhed og ingen klar rollefordeling i systemet.

Der er især to udfordringer.

For det første mangler der overblik og historik. Det er ikke tydeligt, hvem der har ændret hvad, og hvornår det er gjort. Det kan være problematisk, især når det handler om medicin og risikovurderinger, hvor det er vigtigt at kunne dokumentere ændringer.

For det andet er der usikkerhed omkring GDPR. Excel-arket indeholder personfølsomme oplysninger, og regionen har gjort det klart, at navne og identificerbare oplysninger kun må vises i et lukket og godkendt system. Samtidig hænger der en skærm fremme 24 timer i døgnet, hvilket betyder, at information ikke bare kan vises frit.

Derudover sker overlap flere gange dagligt, og alle borgere skal gennemgå systematisk. Når værktøjet ikke er lavet til opgaven, øger det risikoen for fejl eller manglende overlevering.

Der er derfor behov for et system, der er lavet specifikt til formålet:

Et system der understøtter arbejdsgangen ved vagtskifte, sikrer korrekt håndtering af persondata, giver sporbarhed og samtidig er så simpelt, at alle medarbejdere kan bruge det – også dem der ikke er stærke i IT.

1.4 Problemformulering

Hvordan kan der udvikles et digitalt overlapsystem, som erstatter den nuværende Excel-løsning, og som samtidig:

- understøtter arbejdsgangen ved vagtskifte på en struktureret måde
- sikrer korrekt håndtering af medicin (både fast og PN)
- giver sporbarhed og historik over ændringer
- overholder GDPR, herunder krav om anonymisering og rollebaseret adgang
- er så brugervenligt, at selv ikke IT kyndige kan tage fejl af brugen

1.5 1.5 Formål

1.6 1.6 Læsevejledning

2 Business case

2.1 Forretningsmæssig baggrund

Bostedet “Slottet” anvender i dag et Excel-ark til at håndtere overlap mellem vagter, registrering af medicin og opdatering af borgernes risikoprofil. Systemet fungerer som et fælles arbejdsredskab for personalet og bruges flere gange dagligt i forbindelse med vagtskifte (ca. kl. 07, 15 og 23).

Excel-løsningen er ikke udviklet til formålet og mangler funktioner som rollebaseret adgang, sporbarhed og struktureret historik. Samtidig indeholder arket personfølsomme oplysninger, hvilket skaber usikkerhed i forhold til GDPR – især fordi informationer vises på en skærm, der er synlig 24 timer i døgnet.

Der er derfor behov for en løsning, der er udviklet specifikt til bostedets arbejdsgange og som understøtter både drift, dokumentation og lovgivning.

2.2 Forretningsmæssig problemstilling

Den nuværende arbejdsgang medfører følgende problemer:

- Der mangler sporbarhed: Det er uklart, hvem der har foretaget ændringer og hvornår.
- Medicinregistrering sker uden systematisk logning.
- Risikoprofiler skal opdateres mindst tre gange dagligt, men der er ingen automatisk understøttelse af dette krav.
- Personfølsomme oplysninger vises potentielt i et miljø, der ikke er GDPR-mæssigt godkendt.
- Overlap ved vagtskifte er afhængigt af manuelle rutiner og individuel disciplin.

Disse forhold øger risikoen for fejl, manglende dokumentation og usikkerhed omkring databaseskyttelse.

2.3 Interessenter

Ledelsen på bostedet, som er ansvarlig for drift og overholdelse af lovgivning.

Personale og vagtansvarlige, som anvender systemet dagligt i forbindelse med vagtskifte og medicinbehandling.

Regionen, som stiller krav til databeskyttelse og dokumentation.

Borgere, hvis oplysninger behandles i systemet.

2.4 Systemets brugere

Personale

Vagtansvarlig

Leder

Superbruger

2.5 Scenarieoversigt

2.5.1 0-scenariet: Fortsætte med Excel og stramme op

Fordele

- Ingen investering i nyt system
- Ingen større ændring af arbejdsgange

Ulemper

- Fortsat manglende sporbarhed
 - Usikker GDPR-overholdelse
 - Risiko for fejl ved medicinregistrering
 - Vanskeligt at skalere eller dokumentere over for regionen
-

2.5.2 1-scenariet: Udvikling af digitalt overlapsystem (valgt løsning)

Fordele

- Rollebaseret adgang
- Automatisk historik og audit-log
- Anonym visning på public skærm
- Struktur ved vagtskifte
- Bedre dokumentation og GDPR-argumentation

Ulemper

- Udviklingstid
- Kræver ændring af arbejdsgange
- Kræver oplæring af personale

2.5.3 2-scenariet: Manuel forbedring af arbejdsgange uden nyt system

Fordele

- Ingen IT-investering

Ulemper

- Kræver høj disciplin
- Ingen teknisk sikring af sporbarhed
- Vanskeligt at dokumentere compliance

2.5.4 Valgt scenarie

Det foreslås at udvikle et digitalt overlapsystem, som understøtter arbejdsgange ved vagtskifte, medicin håndtering og risikostyring.

2.5.4.1 *Formål*

Formålet med overlapsystemet er:

- at skabe struktur og overblik ved vagtskifte
- at sikre korrekt og dokumenteret medicinhåndtering
- at sikre sporbarhed og historik
- at overholde GDPR via rollebaseret adgang og anonym visning
- at gøre arbejdsgangen enklere og mere robust

2.5.4.2 Forretningsmæssig løsningsbeskrivelse

Den nuværende Excel-løsning erstattes af et system, hvor:

- Borgerstatus registreres struktureret
- Medicin registreres med tidsstempel og bruger-ID
- Risikoprofiler kan opdateres og logges
- Public skærm viser anonymiserede oplysninger
- Historik gemmes som append-only log

Løsningen implementeres internt på bostedet og anvendes af personale og ledelse.

2.5.4.3 It-mæssig løsningsbeskrivelse

Systemet udvikles i .NET MAUI Blazor Hybrid.

Det designes med:

- MVC-arkitektur
- Dependency Injection
- SOLID-principper
- Rollebaseret adgang
- Audit-log

Systemet forberedes til senere udvidelse, men første version fokuserer på kernefunktionalitet.

2.5.4.4 Økonomiske effekter

Der er primært udviklingsomkostninger i form af arbejdstid.

Der forventes ikke væsentlige hardwareinvesteringer, da systemet kan køre på eksisterende udstyr.

På længere sigt kan systemet reducere tidsforbrug ved vagtskifte og mindske risikoen for fejl.

2.5.4.5 *Gevinster*

Interne forbedringer

- Bedre struktur
- Mindre risiko for fejl
- Tydelig ansvarsplacering
- Dokumentation ved tilsyn

Fleksibilitet

Systemet kan senere udvides med yderligere funktionalitet.

2.5.4.6 *Risici*

- Modstand mod nyt system
- IT-ukyndigt personale
- Forkert håndtering af GDPR

Risici håndteres gennem simpelt design, tydelig rolleopdeling og løbende test.

2.5.5 Implementering og opfølgning

2.5.5.1 *Implementeringsstrategi*

Systemet implementeres gradvist.

I en kort periode kan Excel og det nye system køre parallelt for at sikre korrekt overgang.

2.5.5.2 *Gevinstrealisering – KPI'er*

Eksempler på KPI'er:

- Andel af medicinregistreringer med korrekt logning
- Antal manglende risikoprofil-opdateringer

- Tidsforbrug ved vagtskifte
- Antal dokumentationsafvigelser ved tilsyn

3 Scope og Afgrænsning

3.1 In Scope

Projektet omfatter udvikling af et digitalt overlapsystem, der skal:

- understøtte vagtskifte mellem personalegrupper
- give overblik over borgere pr. afdeling
- vise og opdatere risikoprofil (trafiklys)
- registrere fast medicin
- registrere PN medicin med tidspunkt når givet
- give mulighed for at skrive noter og opgaver
- understøtte rollebaseret adgang (personale, vagtansvarlig, leder, superbruger)
- vise anonymiseret information på public skærm
- gemme historik og audit-log over ændringer

3.2 Out of Scope

Projektet omfatter ikke:

- integration med eksterne medicinsystemer
- integration med regionale systemer
- mobilapp-version
- automatisk notifikation via SMS eller mail
- avanceret statistik eller rapportmodul
- integration med eksterne API'er

3.3 Antagelser

3.4 Begrænsninger

4 Metode og Proces

(Den her bliver vigtig til 08-05)

4.1 Overordnet udviklingsmetode

4.2 Iterativ og feature-baseret tilgang

4.2.1 Iterationsstruktur

Udviklingen af VIGOR er organiseret i korte, afgrænsede iterationer med fokus på inkrementel fremdrift og løbende stabilisering af systemet. Hver iteration tager udgangspunkt i én eller få prioriterede use cases fra backloggen og gennemføres som en komplet mini-udviklingscyklus.

En iteration starter med afklaring af scope og konkretisering af den valgte funktionalitet. Herefter vurderes eventuelle konsekvenser for domænemodel, arkitektur og diagrammer. Implementeringen gennemføres i små, kontrollerede skridt, hvor der arbejdes med hyppige commits og løbende build-verificering.

Ved afslutning af iterationen verificeres det, at Definition of Done er opfyldt. Dette indebærer, at systemet bygger uden warnings eller errors, at funktionaliteten er testet manuelt, samt at relevant dokumentation og diagrammer er opdateret ved behov. Iterationen afsluttes med en stabil version i main-grenen.

Denne struktur sikrer sporbarhed mellem analyse, design og implementering og understøtter den iterative tilgang beskrevet i metodeafsnittet.

4.3 Kombination af Scrum og XP-principper

4.4 Artefaktstrategi (levende diagrammer)

4.5 Versionsstyring og commit-strategi

Projektet VIGOR versionsstyres ved anvendelse af Git og GitHub. Formålet med versionsstyringen er at sikre sporbarhed, transparens og mulighed for at genskabe tidligere stabile versioner af systemet gennem hele udviklingsforløbet. Versionsstyringen understøtter den iterative arbejdsform og sikrer, at hver iteration afsluttes i en dokumenteret og stabil tilstand.

Repository'et er struktureret med en hovedgren (main), som altid indeholder en stabil og kørende version af systemet. Udviklingsarbejde udføres i en separat udviklingsgren (dev), hvor nye funktioner og ændringer implementeres og testes, før de eventuelt merges til main. Denne struktur sikrer, at hovedgrenen til enhver tid kan anvendes til demonstration, evaluering eller videreudvikling uden risiko for ustabil kode.

Der arbejdes iterativt, og hver funktionel ændring afsluttes med en commit og push til GitHub. Commit-beskeder formuleres beskrivende og konsekvent for at sikre historisk overblik over udviklingen. Der anvendes en enkel og semantisk inspireret commit-struktur, eksempelvis:

chore: tekniske ændringer uden funktionel påvirkning

feat: tilføjelse af ny funktionalitet

fix: rettelser af fejl

Denne struktur bidrager til klar historik og øget læsbarhed i repository'et.

Da projektet udvikles individuelt, anvendes der ikke formelle Pull Requests eller peer review-processer. I stedet foretages kodegennemgang løbende af udvikleren selv før merge til main-grenen. Inden merge sikres det, at løsningen bygger uden warnings eller errors, og at funktionaliteten er testet manuelt i den aktuelle kontekst. Main-grenen indeholder således altid en stabil og dokumenteret version af systemet.

Ved afslutning af større milepæle oprettes tags i Git, så specifikke stabile versioner kan identificeres og genskabes. Iteration 0 er eksempelvis markeret som en baseline-version, der repræsenterer det tekniske fundament med korrekt projektstruktur, centraliseret Dependency Injection og et clean build.

Samlet set understøtter versionsstyringsstrategien både teknisk kvalitet og dokumentationskravene i semesterprojektet.

4.6 Definition of Done

For at sikre ensartet kvalitet og fremdrift er der fastlagt en fælles Definition of Done (DoD). En opgave betragtes først som afsluttet, når følgende kriterier er opfyldt.

Først og fremmest skal løsningen bygge uden warnings eller errors. Et clean build betragtes som et minimumskrav, da teknisk gæld ikke må videreføres mellem iterationer.

Derudover skal funktionaliteten være testet, som minimum manuelt, for at verificere at den opfører sig som forventet i den aktuelle kontekst. Hvis ændringen påvirker domænemodellen eller arkitekturen, skal relevante diagrammer og dokumentation opdateres, så sporbarheden mellem analyse, design og implementering bevares.

Endelig skal ændringen være committed og pushed til GitHub med en beskrivende commit-besked. Opgaven markeres herefter som afsluttet i Trello og flyttes til "Done"-kolonnen.

Definition of Done fungerer som en kvalitetssikringsmekanisme og understøtter projektets iterative arbejdsform. Den bidrager til at sikre, at hver iteration afsluttes i en stabil og dokumenteret tilstand.

5 Forretningsforståelse og Krav

5.1 Interessenter

- Bostedets personale
- Vagtansvarlige
- Ledelse
- Regionen (indirekte via GDPR-krav)

5.2 Aktører

5.2.1 Personale

Kan registrere medicin, opdatere borgerstatus og skrive noter.

5.2.2 Vagtansvarlig

Kan ændre risikoprofil og fordele opgaver.

5.2.3 Leder

Kan se historik og statistik.

5.2.4 Superbruger

Kan administrere roller og systemopsætning.

5.3 Overordnet forretningsproces (vagtskifte)

5.4 Use case Backlog

Nedenstående use cases udgør systemets samlede backlog. De er prioriteret ud fra forretningsværdi, sikkerhed og lovmæssige krav (herunder medicinhåndtering og GDPR). Hver use case implementeres i en selvstændig iteration.

5.4.1 UC01 – Log ind

Primær aktør: Personale

Beskrivelse: Systemet skal give autoriseret personale adgang via brugernavn og adgangskode og tildele korrekt rollebaseret adgang.

Prioritet: MUST

Iteration: 1

5.4.2 UC02 – Se borgere på afdeling

Primær aktør: Personale

Beskrivelse: Systemet skal vise en liste over borgere på den valgte afdeling med relevante statusoplysninger.

Prioritet: MUST

Iteration: 2

5.4.3 UC03 – Opdatér borgerstatus / risikoprofil

Primær aktør: Personale

Beskrivelse: Systemet skal give mulighed for at opdatere borgerens aktuelle status og risikoprofil i overensstemmelse med gældende krav.

Prioritet: MUST

Iteration: 3

5.4.4 UC04 – Registrér fast medicin

Primær aktør: Personale

Beskrivelse: Systemet skal registrere, at planlagt (fast) medicin er udleveret til borgeren på et givent tidspunkt.

Prioritet: MUST

Iteration: 4

5.4.5 UC05 – Registrér PN medicin

Primær aktør: Personale

Beskrivelse: Systemet skal registrere udlevering af PN-medicin (ved behov), herunder tidspunkt for administration.

Prioritet: MUST

Iteration: 5

5.4.6 UC06 – Se historik / audit-log

Primær aktør: Personale / Leder

Beskrivelse: Systemet skal give adgang til historik over ændringer, medicinregistreringer og statusopdateringer for dokumentation og sporbarhed.

Prioritet: MUST

Iteration: 6

5.4.7 UC07 – Vælg vagt

Primær aktør: Personale

Beskrivelse: Systemet skal give mulighed for at vælge aktiv vagt, så handlinger registreres korrekt i forhold til tidspunkt og ansvar.

Prioritet: SHOULD

Iteration: 7

5.4.8 UC08 – Se overlap

Primær aktør: Personale

Beskrivelse: Systemet skal understøtte visning af relevant information ved vagtskifte for at sikre korrekt overlevering.

Prioritet: SHOULD

Iteration: 8

5.4.9 UC09 – Opret note

Primær aktør: Personale

Beskrivelse: Systemet skal give mulighed for at oprette faglige noter knyttet til en borger.

Prioritet: SHOULD

Iteration: 9

5.4.10 UC10 – Opret og afslut opgave

Primær aktør: Personale

Beskrivelse: Systemet skal give mulighed for at oprette opgaver og markere dem som udført.

Prioritet: SHOULD

Iteration: 10

5.4.11 UC11 – Tildel personale til borger

Primær aktør: Vagtansvarlig

Beskrivelse: Systemet skal give vagtansvarlig mulighed for at fordele personale til specifikke borgere.

Prioritet: SHOULD

Iteration: 11

5.4.12 UC12 – Tildel telefon til personale

Primær aktør: Vagtansvarlig

Beskrivelse: Systemet skal give mulighed for at registrere og fordele arbejdstelefoner til

personale under vagten.

Prioritet: COULD

Iteration: 12

5.4.13 UC13 – Administrér brugere og roller

Primær aktør: Leder

Beskrivelse: Systemet skal give mulighed for at oprette, deaktivere og tildele roller til brugere.

Prioritet: SHOULD

Iteration: 13

5.4.14 UC14 – Public anonym oversigtsskærm

Primær aktør: Offentlig visning (uden login)

Beskrivelse: Systemet skal kunne vise en anonymiseret oversigtsskærm uden personhenførbare oplysninger, egnet til 24/7 visning i fællesområder.

Prioritet: SHOULD

Iteration: 14

5.4.15 UC15 – Statistik og rapporter

Primær aktør: Leder

Beskrivelse: Systemet skal give mulighed for at generere statistisk overblik og rapporter over aktivitet i systemet.

Prioritet: COULD

Iteration: 15

5.5 Use Case Prioriteringstabel

Jeg har prioriteret use cases ud fra, hvilken funktionalitet der skaber størst værdi og sikkerhed for bostedet først. Derfor har jeg placeret medicinhåndtering, risikoprofil og dokumentation øverst, da fejl her kan have direkte konsekvenser for borgernes sikkerhed og for overholdelse af GDPR-krav.

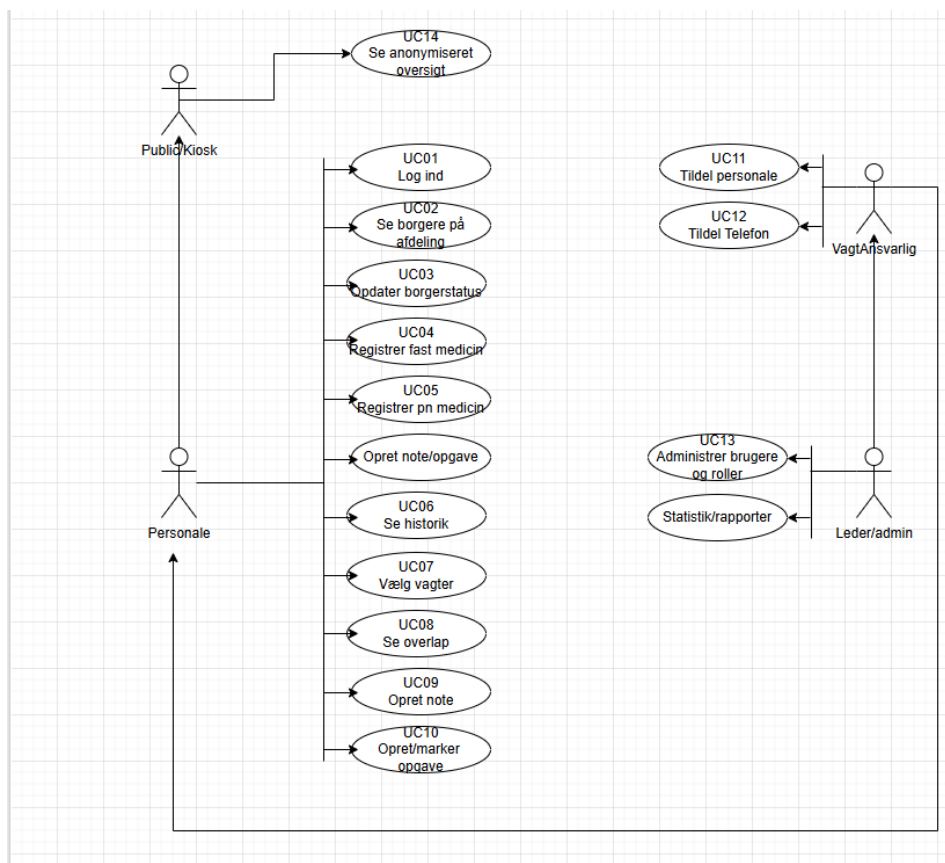
Dernæst har jeg prioriteret funktioner, **der** optimerer arbejdsgangen og de administrative opgaver, da de understøtter driften, men ikke er kritiske for patientsikkerhed eller lovkrav. På den måde sikrer jeg, at systemets kernefunktionalitet leverer reel værdi tidligt i projektet.

UC ID	Use Case	Primær aktør	Forretningsværdi	Risiko ved manglende løsning	Prioritet
UC01	Log ind (rollebaseret)	Personale	Høj	GDPR-brud	MUST
UC02	Se borgere på afdeling	Personale	Høj	Manglende overblik ved overlap	MUST
UC03	Opdatér borgerstatus / risikoprofil	Personale	Meget høj	Forkert risikovurdering	MUST
UC04	Registrér fast medicin	Personale	Meget høj	Medicinfejl / manglende dokumentation	MUST
UC05	Registrér PN medicin	Personale	Meget høj	Akut medicin uden dokumentation	MUST
UC06	Se historik / audit-log	Personale / Leder	Høj	Ingen sporbarhed	MUST
UC07	Vælg vagt	Personale	Høj	Ustruktureret vagtoverlap	SHOULD
UC08	Se overlap	Personale	Høj	Mangelfuld overlevering	SHOULD
UC09	Opret note	Personale	Mellem	Manglende kontekst	SHOULD
UC10	Opret opgave / markér udført	Personale	Mellem	Opgaver glemmes	SHOULD
UC11	Tildel personale til borger	Vagtansvarlig	Mellem	Uklart ansvar	SHOULD
UC12	Tildel telefon til personale	Vagtansvarlig	Lav	Praktisk fordeling	COULD
UC13	Administrér brugere / roller	Leder	Mellem	Uklar adgangsstyring	SHOULD

UC ID	Use Case	Primær aktør	Forretningsværdi	Risiko ved manglende løsning	Prioritet
UC14	Public anonym oversigtsskærm	Public	Mellem	GDPR-udfordring	SHOULD
UC15	Statistik / rapporter	Leder	Lav	Mindre styring	COULD

5.6 Use Case Diagram

Use case-diagrammet viser systemets centrale aktører (Personale, Vagtansvarlig, Leder/Admin og Public) samt deres tilhørende funktioner. Diagrammet illustrerer rollebaseret adgang og giver et samlet overblik over, hvordan overlapsystemet understøtter daglig drift, medicinhandling, vagtskifte og administration.



5.7 Brief Use Cases

UC01 – Log ind (rollebaseret)

Primær aktør: Bruger

Formål: At få adgang til systemet via login og blive sendt videre baseret på rolle.

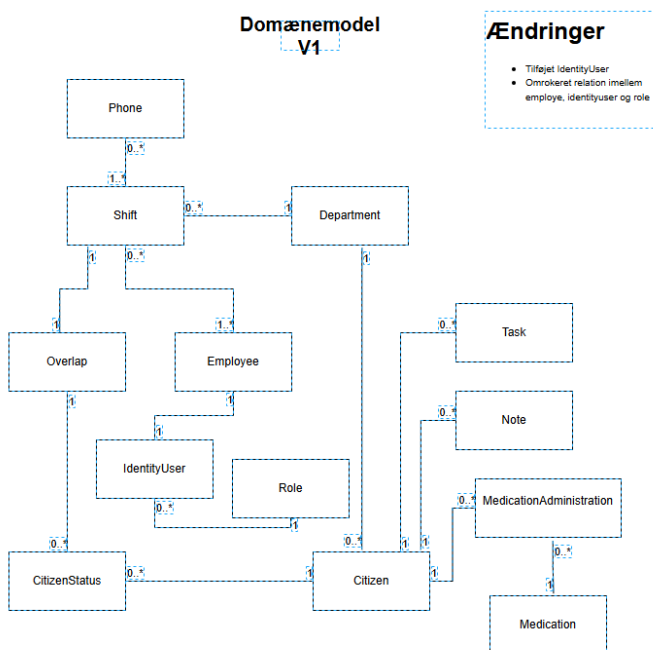
Kort beskrivelse:

Brugeren åbner login-skærmen og indtaster sine loginoplysninger. Systemet validerer oplysningerne via Identity. Hvis login er korrekt, oprettes en session, og brugeren navigeres til næste skærm afhængigt af sin rolle. Hvis login fejler, vises en fejlbesked.

6 Domænenemodell og Analyse

6.1 Domænenemodell (Analysemodel)

Domænenemodellen viser de centrale begreber i overlapsystemets domæne samt relationerne mellem dem. Modellen er konceptuel og indeholder udelukkende forretningsmæssige begreber uden tekniske detaljer. Den illustrerer sammenhængen mellem borgere, vagter (Shift), overlap, statusregistreringer (CitizenStatus), medicinhandling samt medarbejdere og organisatorisk struktur. Formålet er at skabe en fælles forståelse af domænet før design og implementering.



6.2 Begrebsafklaring

6.3 Analyseovervejelser

7 Systemsekvens og Interaktionsanalyse

7.1 System Sequence Diagram – Medicin

7.2 System Sequence Diagram – Vagtskifte

7.3 Analyse af systemgrænse

8 Design og Arkitektur

8.1 Overordnet arkitektur

VIGOR er designet med en lagdelt arkitektur, der adskiller præsentation, forretningslogik og dataadgang. Løsningen består af flere projekter, som hver har et klart ansvarsområde.

MAUI Blazor Hybrid-applikationen fungerer som klient og håndterer brugergrænseflade og interaktion. Web-projektet fungerer som server og eksponerer API-endpoints, hvor forretningslogik og datahåndtering placeres. Shared-projektet indeholder fælles kontrakter, domænemodeller og DTO'er, som deles mellem klient og server.

Arkitekturen understøtter separation of concerns og reducerer koblingen mellem lagene. Præsentationslaget afhænger af abstractions (interfaces) fremfor konkrete implementeringer, hvilket gør systemet mere fleksibelt og testbart.

Strukturen forbereder systemet til fremtidig udvidelse, herunder mulig web-deployment eller integration med eksterne systemer.

8.2 Valg af teknologi (.NET MAUI Blazor Hybrid)

8.3 MVC-struktur

8.4 Dependency Injection

Dependency Injection anvendes konsekvent i hele løsningen for at reducere kobling mellem komponenter og understøtte testbarhed. Services registreres centralt via `ServiceCollectionExtensions` i de respektive projekter, hvilket sikrer en ensartet og overskuelig konfiguration.

Ved at programmere mod interfaces fremfor konkrete implementeringer opnås fleksibilitet i udskiftning af komponenter, eksempelvis ved overgang fra in-memory repository til databasebaseret løsning.

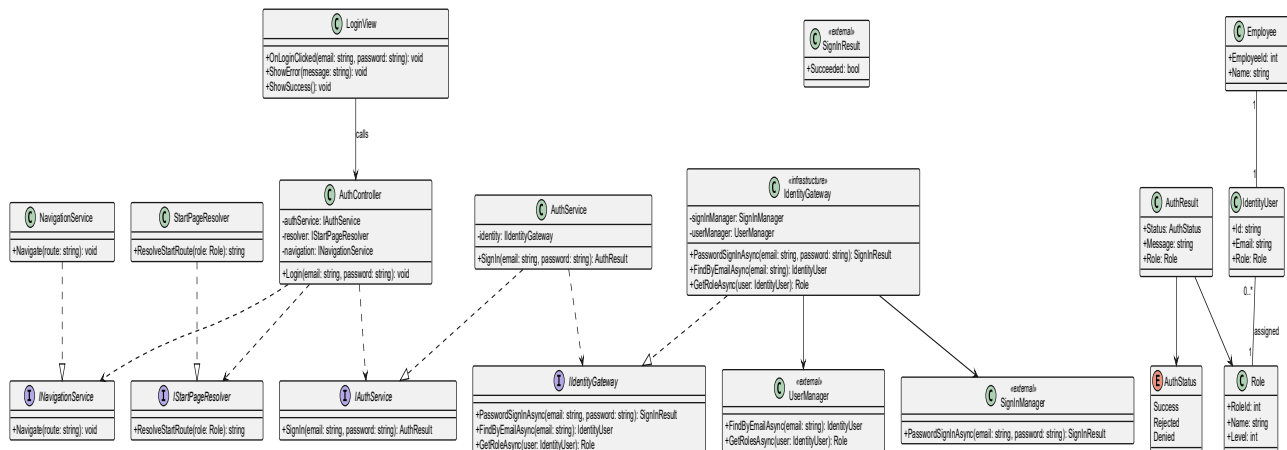
Denne tilgang understøtter SOLID-principperne, særligt Dependency Inversion Principle, og bidrager til en mere vedligeholdbar og skalerbar arkitektur.

8.5 SOLID, GRASP, KISS, DRY

9 Designmodeller

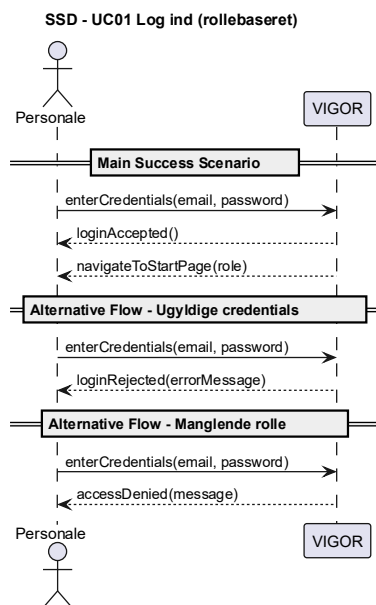
9.1 Design Class Diagram (DCD)

Design class diagrammet anvendes til at beskrive systemets konkrete softwaredesign på klasseniveau. Diagrammet viser klasser, deres ansvar, relationer og afhængigheder. For UC01 illustrerer DCD'en opbygningen efter MVC-princippet samt brugen af interfaces og Dependency Injection for at reducere kobling. DCD'en danner grundlag for implementeringen og sikrer sporbarhed fra analyse til kode.



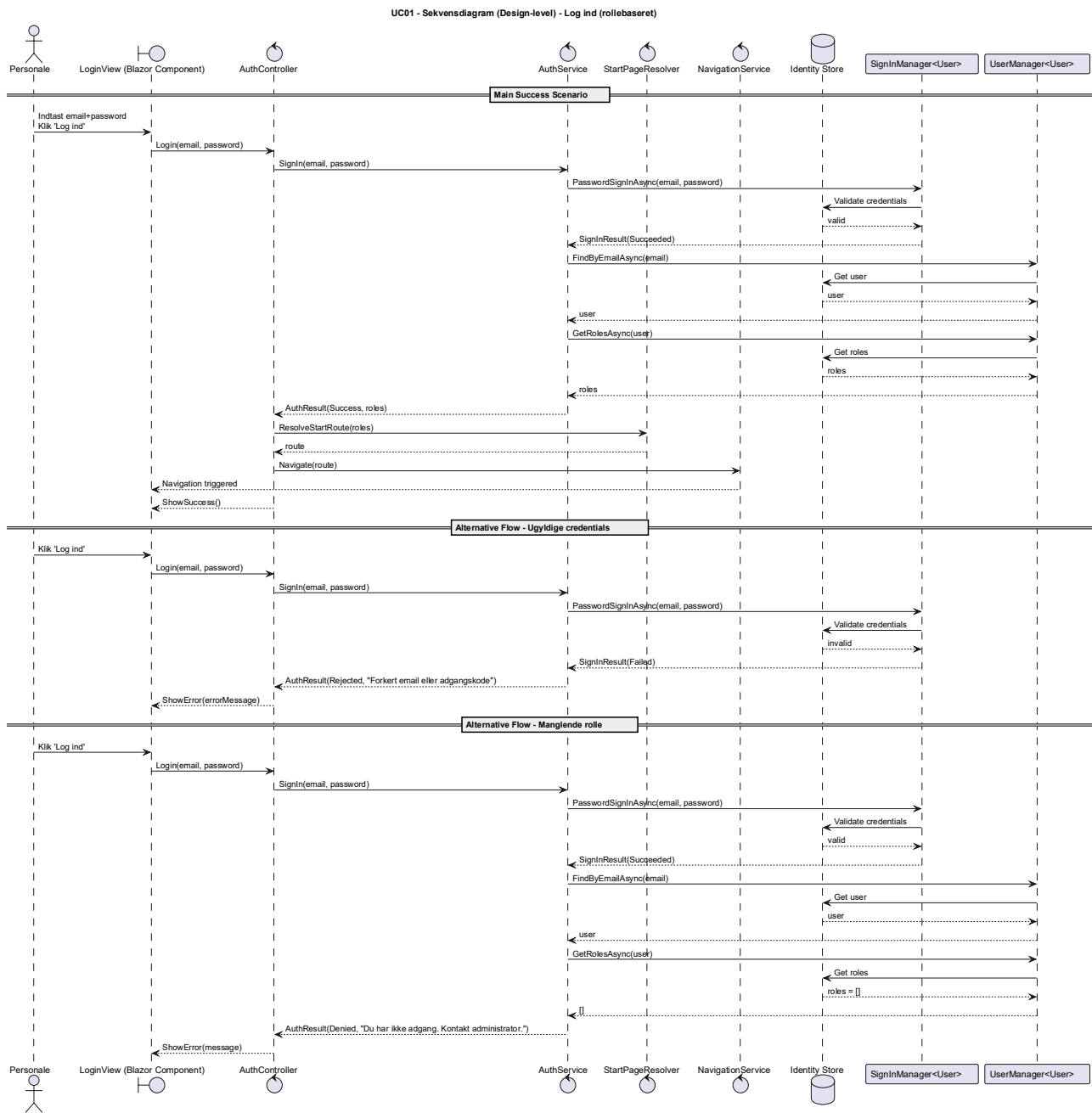
9.2 SSD – UC01 – Login

System Sequence Diagrammet beskriver interaktionen mellem aktøren og systemet. For UC01 viser SSD'en de systemkald, der udføres ved login, samt systemets respons ved succes, ugyldige credentials og manglende rolle. Diagrammet tydeliggør systemets ansvar og grænseflade. SSD'en danner grundlag for det efterfølgende sekvensdiagram.



9.3 Sekvensdiagram – UC01 – Login

Sekvensdiagrammet viser den interne interaktion mellem systemets klasser. Diagrammet viser, hvordan Controller, Services og infrastruktur samarbejder for at gennemføre loginprocessen. Sekvensdiagrammet fungerer som bindeled mellem SSD og Design Class Diagram og understøtter implementeringen.



9.4 Ansvarsfordeling i klasser

10 Datamodel

10.1 ER-diagram

10.2 Entiteter og relationer

10.3 Historik- og audit-strategi

For at sikre sporbarhed og dokumentation er systemet designet med en append-only historikstrategi. Dette betyder, at ændringer ikke overskrives eller slettes, men i stedet gemmes som nye registreringer med tidsstempel og brugerreference.

Denne tilgang understøtter krav om dokumentation i forbindelse med medicinhåndtering og risikoprofilopdateringer. Den bidrager samtidig til GDPR-compliance ved at sikre transparens i forhold til, hvem der har foretaget ændringer og hvornår.

Append-only-princippet reducerer risikoen for manipulation af historiske data og danner grundlag for senere implementering af audit-log og eventuel rapportering.

11 Brugergrænseflade

11.1 Designprincipper

11.2 Wireframe – Public View

11.3 Wireframe – Login

11.4 Wireframe – Overlap screen

11.5 Brugervenlighed for IT-ukyndige

12 Medicinhåndtering

- 12.1 Aktivitetsdiagram – Fast medicin
 - 12.2 PN medicin-flow
 - 12.3 Validering og logning
-

13 Ikke-funktionelle Krav (Supplementary Specification)

- 13.1 Brugervenlighed
 - 13.2 Sikkerhed
 - 13.3 Performance
 - 13.4 Tilgængelighed
 - 13.5 Vedligeholdbarhed
-

14 Risikovurdering og GDPR

- 14.1 GDPR-kontekst
 - 14.2 Risikotabel
 - 14.3 Anonymisering vs. lukket system
 - 14.4 Rollebaseret adgang
 - 14.5 Audit-log og sporbarhed
 - 14.6 Argumentation for løsning
-

15 Deployment og Infrastruktur

VIGOR er designet som et internt system, der implementeres i et lukket miljø på bostedet. Løsningen består af en klientapplikation (MAUI Blazor Hybrid) samt en serverkomponent (ASP.NET Core Web API).

I første version forudsættes lokal installation og drift, uden integration til eksterne systemer. Dette reducerer kompleksitet og sikkerhedsrisici i den indledende fase.

Systemet er arkitektonisk forberedt til senere udvidelse, herunder eventuel web-deployment eller integration med regionale systemer. Rollebaseret adgang og anonymiseret visning sikrer, at systemet kan anvendes både internt og i fællesområder uden at kompromittere persondata.

- 15.1 Deployment diagram
 - 15.2 Lukket installationsscenario
 - 15.3 Ekstern anonym adgang
 - 15.4 Driftsovervejelser
-

16 Implementering

16.1 Projektstruktur

Løsningen er opdelt i flere projekter for at sikre klar ansvarsfordeling og lav kobling mellem komponenter.

VIGOR indeholder MAUI Blazor Hybrid-applikationen og håndterer brugergrænseflade og klientlogik.

VIGOR.Web indeholder serverlogik, API-endpoints og dataadgang.

VIGOR.Shared indeholder fælles kontrakter, domænemodeller og DTO'er, som deles mellem klient og server.

Denne struktur understøtter lagdeling og separation of concerns og gør det muligt at udvikle og teste komponenter isoleret. Projektopdelingen bidrager til vedligeholdbarhed og giver fleksibilitet i forhold til fremtidige ændringer eller udvidelser.

- 16.2 Mappestruktur og namespaces
 - 16.3 Centrale services
 - 16.4 Teststrategi
 - 16.5 Eksempler på kode
-

17 Test og Kvalitet

- 17.1 Unit tests
 - 17.2 Integrationstest
 - 17.3 Manuel test
 - 17.4 Kvalitetssikring
-

18 Refleksion

- 18.1 Metoderefleksion
 - 18.2 Samarbejde og iterationer
 - 18.3 Hvad fungerede godt
 - 18.4 Hvad kunne forbedres
-

19 Konklusion

- 19.1 Opfyldelse af problemformulering
 - 19.2 Fremtidige udvidelser
-

20 Referencer

21 Bilag

21.1 Ekstra diagrammer

21.2 Ekstra use cases

UC01 – Log ind (rollebaseret)

Scope: VIGOR(MAUI Blazor Hybrid)

Primær aktør: Personale

Sekundære aktører: VagtAnsvarlig, Leder/Admin

Interessenter og interesser:

- **Bruger** (Personale/VagtAnsvarlig/Leder/Admin)
 - Ønsker hurtig og sikker adgang til systemet og kun adgang til funktioner, der matcher deres rolle.
- **Bostedet Slottet**
 - Ønsker korrekt adgangskontrol og mulighed for sporbarhed (hvem har været logget ind).
- **Staten / Tilsynsmyndigheder (fx Datatilsynet)**
 - Har interesse i, at systemet overholder gældende lovgivning, herunder GDPR og regler om behandling af personoplysninger i sundheds- og omsorgssektoren.

Systemet: Skal beskytte mod uautoriseret adgang og håndtere fejl robust.

Preconditions:

1. Brugeren har en oprettet konto i Identity.
2. Brugeren er tildelt mindst én rolle.
3. Systemet (app og autentificering) er tilgængeligt.

Minimal Guarantees (ved fejl)

- Ingen session oprettes.
- Brugeren får en forståelig fejlbesked.
- Loginforsøg kan logges til audit/sporbarhed.

Success Guarantees (ved succes)

- Brugeren er autentificeret.
- Brugers rolle(r) er indlæst.
- Brugeren navigeres til korrekt startside baseret på rolle.

Trigger: Brugeren åbner appen og bliver præsenteret for login-skærmen.

Main Success Scenario:

1. Brugeren ser login-skærmen.
2. Brugeren indtaster brugernavn/email og adgangskode.
3. Brugeren vælger "Log ind".
4. Systemet validerer loginoplysninger via Identity.
5. Systemet opretter en autentificeret session.
6. Systemet indlæser brugerens rolle(r).
7. Systemet autoriserer adgang baseret på rolle.
8. Systemet navigerer brugeren til relevant startskærm.

Alternative flows:

E1 – Ugyldige loginoplysninger

- 4a. Identity afviser credentials.
- 4b. Systemet viser fejlbesked: "Forkert brugernavn eller adgangskode."
- 4c. Brugeren forbliver på login-skærmen.

E2 – Konto låst

- 4a. Identity returnerer "locked out".
- 4b. Systemet informerer brugeren om, at kontoen er midlertidigt låst.
- 4c. Brugeren kan ikke logge ind før logout ophører eller admin ændrer status.

E3 – Manglende rolle/adgang

- 6a. Systemet finder ingen gyldig rolle.
- 6b. Systemet afviser adgang og viser besked: "Du har ikke adgang. Kontakt administrator."
- 6c. Ingen adgang gives til systemets funktioner.

E4 – Systemfejl

- 4a. Autentificering fejler på grund af teknisk fejl.
- 4b. Systemet viser generel fejlbesked.
- 4c. Fejlen logges teknisk.

Frekvens: Dagligt, typisk ved start af vagt eller ved genåbning af appen.

21.3 Screenshots

21.4 GitHub link